What is System Design?

Lesson 1 · and the 7-step interview framework

In this lesson → ① The blank whiteboard ② What is system design ③ 1 server → millions ④ What interviewers want ⑤ 7-step framework ⑥ Requirements ⑦ Steps 2–7 ⑧ Common mistakes ⑨ Roadmap

Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

1 What is system design?

> **System design** = choosing the **components**, how they **communicate**, and where **data** lives, so the system meets its requirements *at scale*.

High-Level Design

services · databases
caches · queues · scale

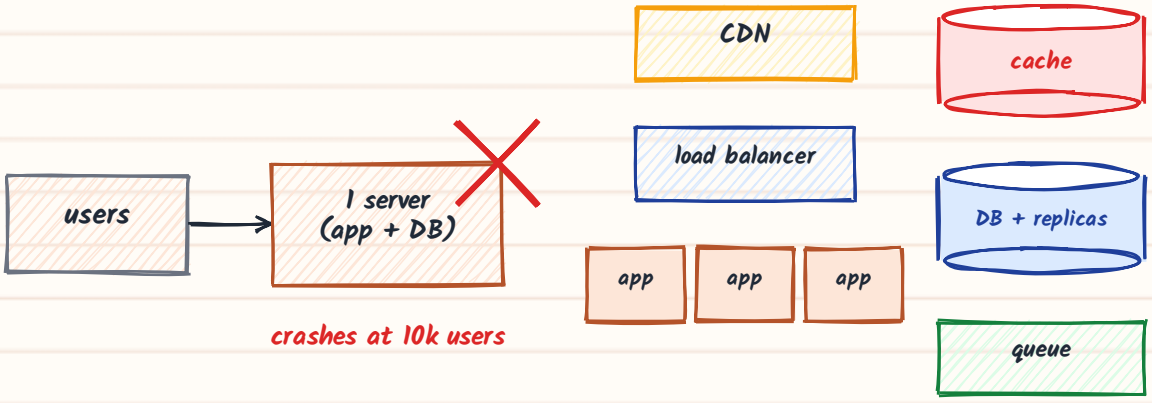
this course ✓

Low-Level Design

classes · interfaces
design patterns

separate course later

2 SnapLink: from 1 server to millions



each box solves one bottleneck

SYSTEM DESIGN NOTEBOOK

Added	Bottleneck it solves	Lesson
Load balancer + app servers	one server can't handle the traffic	SD-03, SD-07
CDN	images far from users, slow	SD-08
Cache	repeated reads hit the DB	SD-15
Read replicas / shards	DB can't keep up	SD-12, SD-13
Queue	slow work blocks requests	SD-18

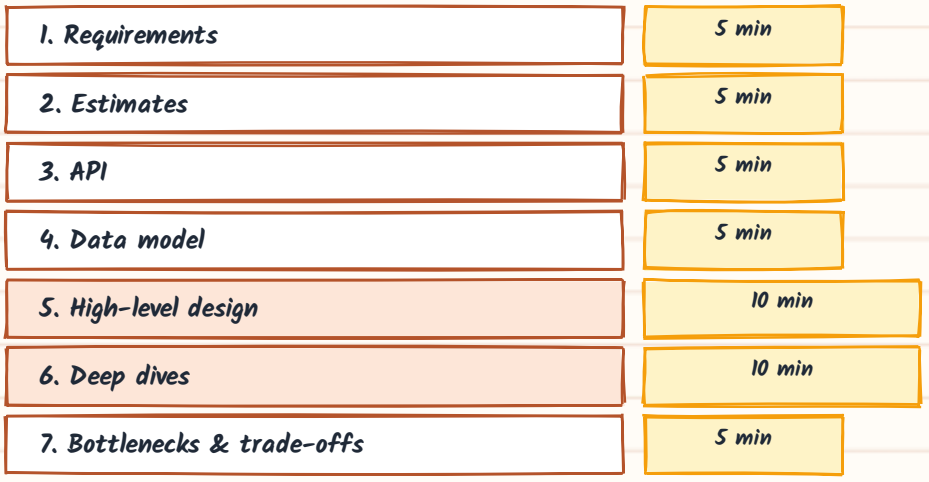
Key idea
Every box in a design must solve a **specific bottleneck**. If you can't say which one, remove it.

3 What interviewers look for

- ✓ **Clarify** before solving
- ✓ **Numbers**: reason about scale
- ✓ **Trade-offs**: why this, not that
- ✓ **Communication**: think out loud, take hints

No single right answer
A simple design, well explained, beats a complex one you can't defend.

4 The 7-step framework (45 min)



Step	Output	Time
1. Requirements	functional + non-functional list	~5 min
2. Estimates	QPS, storage, bandwidth, cache size	~5 min
3. API	3–5 core endpoints	~5 min
4. Data model	entities, DB choice, keys	~5 min
5. High-level design	boxes + one request walked through	~10 min
6. Deep dives	2–3 hardest parts	~10 min
7. Bottlenecks	what breaks at 10×, trade-offs	~5 min

5 Step 1 in practice: SnapLink

- ✓ Upload a photo → 100M users
- ✓ Share a link → Feed < 200 ms (p99)
- ✓ Follow users → 99.99% availability
- ✓ View a feed → Feed may lag a few seconds

Why non-functional matters
"Feed may lag a few seconds" = **eventual consistency is OK** → caching and async fan-out become possible.

6 Steps 2–7 in one line each

Step	SnapLink example
Estimates	10M uploads/day ≈ 115/s; 2 MB each ≈ 7 PB/year
API	POST /photos · GET /feed?cursor= · POST /follow
Data model	User, Photo, Follow → SQL + object storage
High-level design	LB → app servers → cache / DB / object store + CDN
Deep dives	feed generation for celebrities; image upload
Bottlenecks	hot users, DB writes, cache misses at 10×

7 5 common mistakes

Mistake	Do this instead
Jumping to boxes	spend the first 5 min on requirements
No numbers	estimate QPS and storage, then decide
Buzzword soup	add a component only to fix a named bottleneck
Thinking silently	narrate your reasoning; ask questions
Over-engineering	design for the stated scale, then show how to grow

8 Course roadmap

Module	Lessons
1 Foundations	SD-01 – SD-05
2 Networking & edge	SD-06 – SD-09
3 Data & storage	SD-10 – SD-17
4 Async & distributed	SD-18 – SD-22
5 Reliability	SD-23 – SD-24
6 Case studies (12)	SD-25 – SD-36
7 Interview ready	SD-37 – SD-40

Interview Corner

Q. What do you do first in a system design interview?

A. Clarify functional and non-functional requirements, and confirm the scale.

Q. Why do back-of-the-envelope estimates?

A. To justify decisions with numbers: sharding, caching, number of servers.

Q. Is there one right answer?

A. No. Interviewers judge the reasoning, trade-offs and communication.

Q. How do you split 45 minutes?

A. ~20 min requirements, estimates, API, data; ~25 min design and deep dives.

Q. HLD vs LLD?

A. HLD: components, data flow, scale. LLD: classes, interfaces, patterns inside one component.

⚡ Revise in 60 seconds

- → Design = components + communication + data
- → Every box solves a **bottleneck**
- → Clarify → numbers → trade-offs
- → Think out loud
- → **Req · Est · API · Data**
- → **HLD · Deep dives · Trade-offs**
- → ~20 min + ~25 min
- → Avoid the 5 mistakes

Next lesson → SD-02
Back-of-the-envelope estimation: latency numbers, QPS & storage.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**